

SPI_HOST SystemC TLM High-Level Design Specification

1. Introduction

1.1 Purpose

This document provides a high-level SystemC Transaction-Level Modeling (TLM) design specification for the SPI_HOST IP module. The SPI_HOST is a hardware IP that implements a serial peripheral interface (SPI) host controller with advanced features including segment-based command processing, multiple data transfer modes (Standard, Dual, Quad SPI), configurable FIFOs, and comprehensive error detection.

1.2 Scope

This specification covers the transaction-level functional behavior of the SPI_HOST IP, focusing on software-visible features and system-level integration aspects. The document defines:

- Feature classification for TLM modeling scope
- Build-time configuration parameters
- External port interfaces
- Memory-mapped register specifications
- Register callback requirements
- SystemC TLM modeling assumptions

This document explicitly excludes physical layer characteristics, cycle-accurate timing details, and RTL implementation specifics that are not relevant to transaction-level modeling.

1.3 Abstraction Level

The SystemC TLM model operates at the transaction level of abstraction, suitable for software-visible functional verification. The model focuses on:

- Command segmentation and execution as atomic operations
- FIFO management with depth, watermarks, and stall conditions
- Multi-device support via chip select control
- Error detection and interrupt generation for driver validation
- Register-level control and status visibility

The model abstracts away cycle-accurate timing, pin-level signaling, physical and electrical characteristics, and RTL implementation artifacts.

2. Features

2.1 Features to Model in SystemC TLM (IS)

2.1.1 Transaction Control and Command Segmentation

- Segment-based command interface supporting complex multi-phase transactions with different speeds and directions per segment
- Support for TX-only, RX-only, bidirectional, and dummy clock segments within commands
- Command segment queuing with ready/valid handshaking for command submission
- Chip Select Active After Transfer (CSAAT) flag to merge adjacent segments into complex commands
- Software-controlled transaction enable/disable via SPIEN control
- Software reset capability for error recovery and state machine reinitialization

2.1.2 Data Transfer Modes and Capabilities

- Standard SPI mode with full-duplex and half-duplex operation options
- Dual SPI mode supporting 2-bit wide parallel data transfer
- Quad SPI mode supporting 4-bit wide parallel data transfer
- Single Transfer Rate (STR) data sampling on single clock edge
- Arbitrary byte-count support in transactions without alignment restrictions
- Configurable byte ordering (little-endian/big-endian) for multi-byte word transfers

2.1.3 FIFO Management

- Separate TX and RX FIFOs with 288-byte TX capacity and 256-byte RX capacity
- 32-bit word-based FIFO access via memory-mapped registers
- Programmable RX and TX watermark levels for flow control
- FIFO depth status reporting (TXQD, RXQD fields indicating word counts)
- FIFO full/empty status flags for software monitoring
- Automatic FIFO stall detection when TX underflows or RX overflows

2.1.4 Interrupt Generation

- Two interrupt classes: ERROR and SPI_EVENT for condensed interrupt footprint
- Error interrupts covering command errors, FIFO overflow/underflow, invalid command/CSID, and invalid access conditions
- Event interrupts for FIFO watermarks, IDLE, READY, TXEMPTY, RX-FULL states
- Fine-grain interrupt masking through ERROR_ENABLE and EVENT_ENABLE secondary registers

- Interrupt state and enable control for both interrupt classes

2.1.5 Chip Select Control

- Multiple chip select lines (parametrized NumCS) with one-hot active-low encoding
- Per-device configuration through CSID-indexed CONFIGOPTS multi-register
- Automatic chip select assertion/deassertion based on command segments
- CSID switching triggers transaction termination and configuration application

2.1.6 Clock and Timing Configuration

- Programmable clock divider (16-bit CLKDIV field) for per-device SCK frequency control
- SCK frequency derived from core clock with configurable division ratio
- Configurable CPOL (clock polarity) and CPHA (clock phase) for device compatibility
- Full-cycle sampling mode (FULLCYC) option for extended setup time requirements
- Programmable chip select timing margins: CSNIDLE, CSNLEAD, CSNTRAIL parameters

2.1.7 Error Detection and Handling

- Command busy error when writing COMMAND while not READY
- TX FIFO overflow detection and reporting
- RX FIFO underflow detection when reading empty FIFO
- Invalid command segment detection (invalid speed or bidirectional dual/quad mode)
- Invalid CSID detection when command targets non-existent chip select
- Invalid access error for unsupported write patterns to TXDATA
- Error status register with write-one-to-clear semantics
- State machine suspension on unacknowledged errors

2.1.8 Pass-through Mode

- Special mode for direct serial interface control by another module (SPI_DEVICE)
- Pass-through enable signal multiplexes control between SPI_HOST FSM and external block
- Pass-through affects CSB, SCK, and SD signal routing

2.1.9 Status and Control

- Transaction active status indication via ACTIVE field

- Command ready status for software flow control
- Byte order configuration readback for firmware verification
- Output enable control separate from state machine enable
- Stall condition reporting for TX and RX FIFOs during transactions

2.2 Features to Exclude from SystemC TLM (IS NOT)

2.2.1 Physical and Electrical Characteristics

- Voltage levels and current drive specifications (not software-visible)
- Pin input/output impedance characteristics (physical layer detail)
- Signal rise/fall time specifications (analog behavior)
- Pin capacitance and loading effects (electrical property)
- Power consumption and supply voltage requirements (analog/power domain)

2.2.2 Low-Level Timing Specifications

- Exact nanosecond-level setup and hold time requirements (RTL/gate-level concern)
- Clock-to-output delay specifications (physical implementation detail)
- Propagation delay through internal logic stages (not observable at system level)
- Duty cycle requirements and jitter specifications (physical clock properties)
- Minimum/maximum SCK frequency absolute limits (physical limitation, not functional)

2.2.3 Pin-Level Signaling Details

- Tri-state control timing at individual clock cycle boundaries (RTL detail)
- Individual bit transmission timing within byte boundaries (physical layer)
- Signal integrity and noise margin specifications (analog/electrical)
- EMI/EMC compliance characteristics (physical/electromagnetic property)

2.2.4 Unsupported or Excluded Features

- Double Transfer Rate (DTR) mode operation (explicitly not supported)
- Sub-cycle timing control for DTR (not implemented)
- Test modes and boundary scan features (test-only functionality)
- Manufacturing test features (not relevant to functional modeling)

2.2.5 Process and Temperature Dependencies

- Temperature coefficient effects on timing (process-dependent parameter)
- Manufacturing process variation impacts (physical manufacturing detail)
- Aging and reliability characteristics (long-term physical behavior)

2.2.6 Integration-Specific Details

- Physical pin multiplexing with other peripherals (system integration concern)
- Board-level routing constraints and trace impedance (external to IP)
- External pull-up/pull-down resistor requirements (board-level design)
- Specific flash device vendor compatibility details (target device specific, not host feature)

2.2.7 Reserved and Unimplemented Functionality

- Reserved bits in configuration registers (not functional)
- Unimplemented configuration options (not available)
- Future expansion fields (not currently active)

2.3 Feature Summary

- **Total IS Features:** 10 major categories covering transaction control, data transfer modes, FIFOs, interrupts, chip selects, clocking, error handling, pass-through mode, and status/control
- **Total IS NOT Features:** 8 major categories covering physical/electrical properties, low-level timing, pin-level signaling, unsupported features, process dependencies, integration details, and reserved functionality

The IS features represent the software-visible, functionally significant aspects of the SPI_HOST that affect driver behavior and system-level transaction modeling. The IS NOT features represent implementation details, physical characteristics, and RTL/gate-level concerns that are not necessary for transaction-level modeling.

3. Configuration Parameters

3.1 Build-Time Configuration Parameters

These parameters are static and fixed at hardware instantiation time, not runtime-programmable registers. They affect SystemC TLM model behavior and must be configurable during model instantiation.

Parameter Name	Type	Default Value	Valid Range	Description
NumCS	unsigned int	1	1-32	Number of chip select lines (CSB[NumCS-1:0]) supported by the SPI host. Determines how many SPI devices can be independently controlled. Affects CSID register validation range and CSB line instantiation.
ByteOrder	bool	1 (Little-Endian)	0 or 1	Controls byte ordering within 32-bit words for TX-DATA/RXDATA. 0=Big-Endian (MSB transmitted first), 1=Little-Endian (LSB transmitted first). Must match host processor endianness for correct data ordering.

Parameter Name	Type	Default Value	Valid Range	Description
TxDpth	unsigned 72 int		1-1024	TX FIFO depth in 32-bit words. Controls transmit buffer capacity. The TX FIFO is 36 bits wide internally (32 data + 4 byte-enable). Affects FIFO status, watermark behavior, and transaction stall conditions. Default 72 words = 288 bytes.
RxDpth	unsigned 64 int		1-1024	RX FIFO depth in 32-bit words. Controls receive buffer capacity. Affects FIFO status, watermark behavior, and transaction stall conditions. Default 64 words = 256 bytes.

3.2 Parameter Classification Rationale

3.2.1 Included Parameters These parameters meet all required criteria: - Vary across products/instantiations of the IP - Change functional behavior in SystemC/TLM models - Fall inside the “Is” features scope (modeling scope) -

Are unique/relevant to this specific SPI Host datasheet

NumCS (Number of Chip Select Lines): - Varies: Different SoC implementations may need 1, 2, or more chip select lines - Functional Impact: Determines CSB signal array size, CSID validation range, and multi-device support - In Scope: Affects chip select control feature, which is in the “Is” list - Unique: Specific to this SPI Host implementation’s multi-device capability

ByteOrder (Byte Ordering for Multi-Byte Words): - Varies: Must match processor architecture (Little-Endian for Ibex, could be Big-Endian for other CPUs) - Functional Impact: Changes byte transmission/reception order in TXDATA/RXDATA, affecting data correctness - In Scope: Affects data transfer modes and FIFO management features in the “Is” list - Unique: Specific configuration for this SPI Host’s processor integration

TxDpth (TX FIFO Depth): - Varies: Can be configured based on performance vs. area tradeoffs (default 72 words = 288 bytes) - Functional Impact: Determines TX buffer capacity, watermark behavior, TXQD status, and stall conditions - In Scope: Core to FIFO management feature in the “Is” list - Unique: Parameterized compile-time value for this SPI Host implementation

RxDpth (RX FIFO Depth): - Varies: Can be configured based on performance vs. area tradeoffs (default 64 words = 256 bytes) - Functional Impact: Determines RX buffer capacity, watermark behavior, RXQD status, and stall conditions - In Scope: Core to FIFO management feature in the “Is” list - Unique: Parameterized compile-time value for this SPI Host implementation

3.2.2 Parameters Excluded Runtime-Programmable Features (Not Build-Time Parameters): - CPOL, CPHA, FULLCYC: Clock polarity, phase, and full-cycle sampling - configured via CONFIGOPTS register per device - CLKDIV: Clock divider ratio - runtime programmable via CONFIGOPTS.CLKDIV (16-bit field) - CSNLEAD, CSNTRAIL, CSNIDLE: Chip select timing margins - runtime configurable via CONFIGOPTS register - TX_WATERMARK, RX_WATERMARK: FIFO watermark levels - runtime programmable via CONTROL register - SPIEN: IP enable/disable - runtime controlled via CONTROL.SPIEN - Interrupt enables: ERROR_ENABLE, EVENT_ENABLE, INTR_ENABLE - all runtime programmable - Command segments: DIRECTON, SPEED, CSAAT, LEN - specified per segment via COMMAND register - CSID: Chip select ID - runtime selectable via CSID register

Fixed Implementation Characteristics (Not Configurable Parameters): - Command FIFO depth: Fixed capacity for one additional segment descriptor - not parameterized - TL-UL bus width: Fixed at 32 bits - standard bus interface specification - Shift register width: Fixed at 8 bits - internal implementation detail - SD bus width: Fixed at 4 bits (SD[3:0]) - defined by SPI Quad protocol specification - Support for Standard/Dual/Quad modes: Fixed functional capability - not optional - Single Transfer Rate (STR) only: Fixed - no DTR

support in this IP - Pass-through mode support: Fixed functional capability - not optional

3.3 SystemC TLM Modeling Implications

For SystemC TLM implementation, these parameters should be:

1. **Constructor Parameters:** Passed during IP instantiation to configure model behavior
2. **IP-XACT Properties:** Exposed as configurable properties in IP-XACT descriptions for integration tools
3. **Behavioral Impact:** Must affect model functionality including:
 - **NumCS:** CSB signal array instantiation, CSID validation (0 to NumCS-1), chip select control logic
 - **ByteOrder:** Byte ordering in TXDATA writes and RXDATA reads, data packing/unpacking logic
 - **TxDpth:** TX FIFO sizing, TXQD status calculation, TX-FULL/TXEMPTY flags, TX_WATERMARK comparison
 - **RxDpth:** RX FIFO sizing, RXQD status calculation, RX-FULL/RXEMPTY flags, RX_WATERMARK comparison
4. **Status Register Visibility:**
 - **ByteOrder:** Reflected in STATUS.BYTEORDER (bit 22) for firmware readback
 - **TxDpth:** Affects STATUS.TXQD (bits 7:0) maximum value and TXFULL/TXEMPTY thresholds
 - **RxDpth:** Affects STATUS.RXQD (bits 15:8) maximum value and RXFULL/RXEMPTY thresholds
 - **NumCS:** Affects CSID validation and CSB signal behavior

4. Port Interfaces

4.1 External Port Interface Specifications

Interface Category	Port Name	Port Type	Direction	Width	Description
Register Bus	reg_bus	tlm_target	Target<32>	32-bit	TL-UL register bus interface for configuration and status register access

Interface Category	Port Name	Port Type	Direction	Width	Description
Protocol Transaction	spi_master	sc_port	Initiator	N/A	SPI master transaction interface for initiating Standard/Dual/Quad SPI commands to target devices
Protocol Transaction	passthrough_sc_export	sc_export	Export	N/A	Pass-through request interface from SPI_DEVICE for direct serial interface control
Protocol Transaction	passthrough_sc_inport	sc_inport	Initiator	N/A	Pass-through response interface to SPI_DEVICE for received data during pass-through mode
Interrupt	error_irq	sc_out	Output	1-bit	Error interrupt output for programming violations and error conditions

Interface Category	Port Name	Port Type	Direction	Width	Description
Interrupt	spi_event_irq_out	sc_out	Output	1-bit	Event interrupt output for FIFO watermarks, IDLE, READY, and transaction completion events
DMA Request	dma_trigger	sc_out	Output	1-bit	DMA trigger output signal asserted when RX or TX FIFOs reach configured watermark levels
Clock	clk_i	sc_in	Input	N/A	Functional clock frequency input for SPI SCK generation and internal FSM operation (Hz)
Reset	rst_ni	sc_in	Input	1-bit	Active-low asynchronous reset input for hardware initialization

4.2 Port Interface Descriptions

4.2.1 Register Bus Interface (reg_bus) The TL-UL (Tilelink Uncached Lightweight) register bus interface provides 32-bit memory-mapped access to all configuration, control, status, and data registers. This interface supports standard read/write transactions with byte-enable support for sub-word accesses.

4.2.2 SPI Master Protocol Interface (spi_master) The SPI master transaction interface abstracts the physical SPI signals (SCK, CSB, SD) into transaction-level method calls. This interface supports: - Standard SPI (1-bit data transfer) - Dual SPI (2-bit data transfer) - Quad SPI (4-bit data transfer) - Configurable clock polarity (CPOL) and phase (CPHA) - Variable-length transactions with byte granularity

4.2.3 Pass-through Interfaces (passthrough_in, passthrough_out) The pass-through interfaces enable a special mode where control of the SPI signals is transferred from the SPI_HOST FSM to an external SPI_DEVICE module. These interfaces support: - Request path (passthrough_in): Control signals from SPI_DEVICE - Response path (passthrough_out): Data forwarding to SPI_DEVICE - CSB[0] signal routing when pass-through is active

4.2.4 Interrupt Outputs (error_irq, spi_event_irq) Two independent interrupt outputs provide: - **error_irq**: Asserted for error conditions (CMDDBUSY, OVERFLOW, UNDERFLOW, CMDINVAL, CSIDINVAL, ACCESSINVAL) - **spi_event_irq**: Asserted for event conditions (IDLE, READY, TXEMPTY, RXFULL, TXWM, RXWM)

Both interrupts support fine-grain masking through secondary enable registers.

4.2.5 DMA Trigger Output (dma_trigger) The DMA trigger output enables efficient data transfers by signaling when: - TX FIFO level falls below TX_WATERMARK (needs more data) - RX FIFO level reaches or exceeds RX_WATERMARK (has data available)

4.2.6 Clock and Reset (clk_i, rst_ni)

- **clk_i**: Functional clock frequency input used to calculate SPI SCK frequency based on CLKDIV settings
- **rst_ni**: Active-low reset initializes all internal state machines, FIFOs, and registers to their default values

5. Memory-Mapped Registers

5.1 Register Summary

Register Name	Offset	Size (bits)	Access	Reset Value	Description
INTR_STATE	0x0	32	RW1C/RO	0x0	Interrupt state register. error field is RW1C, spi_event is RO
INTR_ENABLE	0x4	32	RW	0x0	Interrupt enable register for error and spi_event interrupts
INTR_TEST	0x8	32	WO	0x0	Interrupt test register to force interrupt conditions
ALERT_TEST	0xC	32	WO	0x0	Alert test register for fatal_fault alert testing
CONTROL	0x10	32	RW	0x7F	Control register for SPIEN, SW_RST, OUT-PUT_EN, and FIFO watermarks
STATUS	0x14	32	RO	0x0	Status register for READY, ACTIVE, FIFO levels, and operational flags

Register Name	Offset	Size (bits)	Access	Reset Value	Description
CONFIGOPTS	0x18	32	RW	0x0	Configuration options for clock divider, timing, CPOL, CPHA, FULLCYC
CSID	0x1C	32	RW	0x0	Chip select ID to target specific SPI device
COMMAND	0x20	32	WO	0x0	Command segment descriptor with LEN, DIRECTION, SPEED, CSAAT fields
RXDATA	0x24	32	RO	N/A	SPI receive data FIFO read window
TXDATA	0x28	32	WO	N/A	SPI transmit data FIFO write window, supports byte-enables
ERROR_ENABLE	0x2C	32	RW	0x1F	Controls which error classes trigger error interrupt

Register Name	Offset	Size (bits)	Access	Reset Value	Description
ERROR_STATUS	0x30	32	RW1C	0x0	Error status flags for programming violations, must be cleared before proceeding
EVENT_ENABLE	0x34	32	RW	0x0	Controls which SPI events trigger spi_event interrupt

5.2 Detailed Register Descriptions

5.2.1 Interrupt Registers INTR_STATE (0x0): Contains error and spi_event interrupt state bits. The error bit is write-1-to-clear, while spi_event is read-only status.

INTR_ENABLE (0x4): Enables error and spi_event interrupts at the top level. Both fields are read-write.

INTR_TEST (0x8): Write-only register for testing interrupt generation by forcing interrupt states.

ALERT_TEST (0xC): Write-only register to trigger fatal_fault alert for testing purposes.

5.2.2 Control and Status Registers CONTROL (0x10): - SPIEN (bit 31): Enables SPI_HOST state machine - SW_RST (bit 30): Software reset for internal state and FIFOs - OUTPUT_EN (bit 29): Enables output buffers for SCK, CSB, and SD signals - TX_WATERMARK (bits 15:8): TX FIFO watermark level in 32-bit words - RX_WATERMARK (bits 7:0): RX FIFO watermark level in 32-bit words, default 0x7F

STATUS (0x14): Read-only register providing operational status: - READY (bit 31): Host ready to accept new command segment - ACTIVE (bit 30): Host actively processing command - TXFULL (bit 29): TX FIFO full - TXEMPTY (bit 28): TX FIFO empty - TXSTALL (bit 27): Transaction stalled due to TX FIFO underflow - TXWM (bit 26): TX FIFO below watermark level - RXFULL (bit 25): RX FIFO full - RXEMPTY (bit 24): RX FIFO empty - RXSTALL (bit 23): Transaction stalled due to RX FIFO overflow - BYTEORDER (bit 22):

ByteOrder parameter value readback - RXWM (bit 20): RX FIFO at or above watermark level - CMDQD (bits 19:16): Command queue depth in segments - RXQD (bits 15:8): RX FIFO depth in 32-bit words - TXQD (bits 7:0): TX FIFO depth in 32-bit words

5.2.3 Configuration Registers CONFIGOPTS (0x18): Per-device configuration parameters: - CPOL (bit 31): Clock polarity (0=idle low, 1=idle high) - CPHA (bit 30): Clock phase (0=sample on leading edge, 1=sample on trailing edge) - FULLCYC (bit 29): Full-cycle sampling mode for extended setup time - CSNLEAD (bits 27:24): Half-cycles between CSB fall and first SCK edge - CSNTRAIL (bits 23:20): Half-cycles between last SCK edge and CSB rise - CSNIDLE (bits 19:16): Half-cycles to hold CSB high between commands - CLKDIV (bits 15:0): Clock divider, SCK period = $2(CLKDIV+1)T_{core}$

CSID (0x1C): 32-bit chip select identifier. Selects which CSB line to assert during command execution. Valid range is 0 to NumCS-1.

5.2.4 Command and Data Registers COMMAND (0x20): Write-only command segment descriptor: - LEN (bits 24:5): Segment length in bytes minus 1 (0-1048575) - DIRECTION (bits 4:3): 0=Dummy, 1=RX-only, 2=TX-only, 3=Bidirectional - SPEED (bits 2:1): 0=Standard, 1=Dual, 2=Quad, 3=Reserved - CSAAT (bit 0): Chip Select Active After Transaction (0=deassert, 1=keep active)

RXDATA (0x24): Read-only 32-bit window to RX FIFO. Reading pulls data from FIFO. Byte order depends on ByteOrder parameter.

TXDATA (0x28): Write-only 32-bit window to TX FIFO. Supports byte-enable for partial word writes. Byte order depends on ByteOrder parameter.

5.2.5 Error and Event Registers ERROR_ENABLE (0x2C): Mask register for error interrupt sources, default all enabled (0x1F): - CSIDINVAL (bit 4): Invalid CSID value (exceeds NumCS) - CMDINVAL (bit 3): Invalid command segment parameters - UNDERFLOW (bit 2): RX FIFO underflow (read from empty FIFO) - OVERFLOW (bit 1): TX FIFO overflow (write to full FIFO) - CMDBUSY (bit 0): Command written while STATUS.READY=0

ERROR_STATUS (0x30): Write-1-to-clear error status flags: - ACCESSINVAL (bit 5): Invalid TLUL byte-enable pattern (cannot be masked) - CSIDINVAL (bit 4): Command with invalid CSID detected - CMDINVAL (bit 3): Invalid SPEED or bidirectional dual/quad command - UNDERFLOW (bit 2): Read from empty RX FIFO - OVERFLOW (bit 1): TX FIFO overflow - CMDBUSY (bit 0): Command written when not READY

EVENT_ENABLE (0x34): Mask register for event interrupt sources: - IDLE (bit 5): Interrupt when STATUS.ACTIVE goes low - READY (bit 4): Interrupt when STATUS.READY goes high - TXWM (bit 3): Interrupt when TX FIFO falls below watermark - RXWM (bit 2): Interrupt when RX FIFO

reaches watermark - TXEMPTY (bit 1): Interrupt when TX FIFO becomes empty - RXFULL (bit 0): Interrupt when RX FIFO becomes full

5.3 Register Access Legend

- **RO**: Read Only
 - **RW**: Read/Write
 - **WO**: Write Only
 - **RW1C**: Write-1-to-Clear
-

6. Register Callbacks

6.1 Registers Requiring Write Callbacks

Callback Name	Description
handle_write_INTR_STATE	W1C for error bit clears error interrupt state, may de-assert error_irq port if INTR_ENABLE.error is set and no other errors remain
handle_write_INTR_TEST	Forces interrupt state bits: sets INTR_STATE.error or INTR_STATE.spi_event, asserts error_irq or spi_event_irq ports based on INTR_ENABLE settings
handle_write_ALERT_TEST	Triggers fatal_fault alert signal for testing purposes
handle_write_CONTROL	SPIEN enables/disables FSM and resumes/suspends transactions; SW_RST resets FIFOs, CDC, FSM, and shift register; OUTPUT_EN enables/disables output buffers for SCK, CSB, SD; TX_WATERMARK/RX_WATERMARK affect watermark event interrupt generation via spi_event_irq and dma_trigger ports

Callback Name	Description
handle_write_COMMAND	Queues command segment to FSM, initiates SPI transaction via spi_master port, checks STATUS.READY dependency (rejects if not ready and sets ERROR_STATUS.CMDBUSY), validates CSID against NumCS parameter (sets ERROR_STATUS.CSIDINVAL if invalid), validates SPEED and DIRECTION fields (sets ERROR_STATUS.CMDINVAL if invalid or bidirectional dual/quad), updates STATUS.READY and STATUS.ACTIVE flags
handle_write_TXDATA	Pushes data to TX FIFO, updates STA-TUS.TXQD/TXFULL/TXEMPTY/TXWM, validates byte-enable mask (sets ERROR_STATUS.ACCESSINVAL and halts FSM for invalid patterns like zero-byte writes), checks TX FIFO full condition (sets ERROR_STATUS.OVERFLOW and halts FSM if full), may trigger spi_event_irq if TXWM condition met and EVENT_ENABLE.TXWM set, may assert dma_trigger if TX watermark reached
handle_write_ERROR_STATUS	W1C clears error bits, resumes FSM operation if all errors cleared, may de-assert error_irq port if no errors remain active
handle_write_INTR_ENABLE	Enables/disables error and spi_event interrupt propagation to error_irq and spi_event_irq ports based on current INTR_STATE values
handle_write_ERROR_ENABLE	Configures error interrupt masking, affects which error conditions trigger error_irq port assertions

Callback Name	Description
handle_write_EVENT_ENABLE	Configures event interrupt sources (RXFULL, TXEMPTY, RXWM, TXWM, READY, IDLE), affects spi_event_irq and dma_trigger port assertions based on STATUS register conditions

6.2 Registers Requiring Read Callbacks

Callback Name	Description
handle_read_STATUS	Returns live hardware status reflecting current FSM state (READY, ACTIVE), real-time FIFO depths (TXQD, RXQD, CMDQD), FIFO flags (TXFULL, TXEMPTY, RXFULL, RXEMPTY), watermark flags (TXWM, RXWM), stall conditions (TXSTALL, RXSTALL), and ByteOrder parameter value
handle_read_RXDATA	Pops data from RX FIFO, updates STA-TUS.RXQD/RXFULL/RXEMPTY/RXWM, checks RX FIFO empty condition (sets ER-ROR_STATUS.UNDERFLOW and halts FSM if empty), may trigger spi_event_irq if RXWM condition changes and EVENT_ENABLE.RXWM set, may de-assert dma_trigger if RX watermark condition clears

6.3 Callback Summary

Total Registers Requiring Callbacks: 12 - **Write-only callbacks:** 10 registers - **Read-only callbacks:** 2 registers - **Both read and write callbacks:** 0 registers

6.4 Key Callback Categories

6.4.1 Immediate Functional Side-Effects **State/Mode Changes:** - CONTROL (SPIEN, SW_RST, OUTPUT_EN) - COMMAND (FSM state transi-

tions) - ERROR_STATUS (FSM resume)

FIFO Operations: - TXDATA (TX FIFO push) - RXDATA (RX FIFO pop)

Interrupt Assertions: - INTR_TEST (force interrupts) - INTR_STATE (W1C clears) - INTR_ENABLE (enable/disable propagation) - ERROR_ENABLE (error masking) - EVENT_ENABLE (event masking)

Port Triggering: - CONTROL (dma_trigger) - TXDATA (dma_trigger) - RXDATA (dma_trigger) - All interrupt-related registers (error_irq, spi_event_irq)

Protocol Actions: - COMMAND (SPI transaction initiation via spi_master port)

Alert Generation: - ALERT_TEST (fatal_fault alert)

6.4.2 Conditional Access Dependencies

- **COMMAND write dependency:** Requires STATUS.READY=1, otherwise sets ERROR_STATUS.CMDBUSY and halts FSM
- **COMMAND CSID validation:** Requires CSID < NumCS parameter, otherwise sets ERROR_STATUS.CSIDINVAL and halts FSM
- **COMMAND field validation:** Requires valid SPEED (0-2) and valid DIRECTION for dual/quad modes, otherwise sets ERROR_STATUS.CMDINVAL and halts FSM
- **TXDATA write dependency:** Requires TX FIFO not full, otherwise sets ERROR_STATUS.OVERFLOW and halts FSM; requires valid byte-enable patterns (byte, half-word, or full-word), otherwise sets ERROR_STATUS.ACCESSINVAL and halts FSM
- **RXDATA read dependency:** Requires RX FIFO not empty, otherwise sets ERROR_STATUS.UNDERFLOW and halts FSM
- **ERROR_STATUS must be cleared:** Before FSM can resume operations and accept new commands after any error condition

6.4.3 Abstract Port Mappings **Interrupt Ports:** - error_irq: Controlled by INTR_STATE.error, INTR_ENABLE.error, INTR_TEST.error, ERROR_STATUS, ERROR_ENABLE - spi_event_irq: Controlled by INTR_STATE.spi_event, INTR_ENABLE.spi_event, INTR_TEST.spi_event, EVENT_ENABLE, STATUS flags

DMA Trigger Port: - dma_trigger: Asserted when TX FIFO below TX_WATERMARK or RX FIFO at/above RX_WATERMARK

Protocol Transaction Port: - spi_master: Transaction initiated by COMMAND writes, controlled by CONTROL.SPIEN and CONTROL.OUTPUT_EN

Alert Port: - fatal_fault: Triggered by ALERT_TEST writes

6.5 Registers NOT Requiring Callbacks

The following registers are pure storage with no immediate side-effects and no access dependencies: - **CONFIGOPTS**: Pure configuration storage, values applied when COMMAND references CSID - **CSID**: Pure storage, value validated when COMMAND is written

7. Modeling Assumptions

7.1 Abstraction Level and Timing

1. **Transaction-level modeling**: Commands processed as atomic transactions, not pin-level or cycle-accurate behavior
2. **SCK clock abstraction**: Clock generation timing expressed via CLKDIV, CPOL, CPHA parameters; no actual clock signal toggling modeled
3. **Timing parameters as delays**: CSNLEAD, CSNTRAIL, CSNIDLE modeled as transaction delays, not half-cycle accurate waveforms
4. **FULLCYC sampling mode**: Implemented as functional choice affecting data capture, not detailed timing simulation
5. **FSM state transitions abstracted**: State machine behavior captured at command segment boundaries, not individual clock cycles

7.2 Command Segmentation and Control

6. **Segment-based transaction model**: Each COMMAND register write creates discrete segment transaction with DIRECTION, SPEED, LEN, CSAAT fields
7. **CSAAT flag behavior**: Modeled as transaction continuation indicator; CSB assertion/deassertion tracked logically, not as signal edges
8. **Command FIFO depth**: Fixed capacity for one additional segment descriptor; modeled as simple queuing without detailed CDC implementation
9. **Ready/valid handshaking**: Represented at transaction boundaries via STATUS.READY flag, not cycle-by-cycle protocol
10. **Multi-segment command composition**: Commands composed from sequential segment transactions; inter-segment timing abstracted

7.3 Data Transfer and Speed Modes

11. **Standard/Dual/Quad mode abstraction**: Modeled as data width multipliers (1x, 2x, 4x throughput) without pin-level SD bus simulation
12. **SD bus impedance control**: Not modeled; direction changes handled at segment boundaries without tri-state simulation
13. **Bidirectional mode limitation**: Only supported for Standard mode per specification; Dual/Quad bidirectional generates CMDINVAL error
14. **Byte ordering logic**: ByteOrder parameter determines word packing/unpacking order in TXDATA/RXDATA, not bit-level shifting

15. **Single Transfer Rate only:** STR mode modeled; DTR (Dual Transfer Rate) explicitly excluded

7.4 FIFO Management

16. **TX FIFO depth:** Parameterized as TxDepth words (default 72 words = 288 bytes); actual 36-bit width (32 data + 4 byte-enable) transparent to TLM
17. **RX FIFO depth:** Parameterized as RxDepth words (default 64 words = 256 bytes); 32-bit word-aligned storage
18. **FIFO status flags:** TXQD/RXQD counts, TXFULL/TXEMPTY/RXFULL/RXEMPTY flags updated at transaction boundaries
19. **Watermark behavior:** RX_WATERMARK/TX_WATERMARK compared against queue depth for event interrupt generation
20. **Stall conditions:** TXSTALL/RXSTALL modeled when FIFO underflow/overflow detected; transaction suspended until resolved
21. **Byte Select N+1 depth effect:** TX FIFO effective depth of N+1 incorporated into status signals; internal buffering transparent
22. **Transient stalls:** Implementation-specific delays from byte packing/unpacking not modeled in detail; only persistent stall conditions tracked

7.5 Chip Select and Device Management

23. **NumCS parameter:** Determines CSB signal array size and CSID validation range; each device independently configurable
24. **CSID-indexed configuration:** CONFIGOPTS multi-register accessed by CSID value; configuration changes applied only when CSB deasserted
25. **One-hot chip select encoding:** CSB lines asserted active-low, one at a time; multi-device arbitration not modeled
26. **Configuration switching delay:** Extended idle time when CONFIGOPTS changes modeled as transaction delay, not detailed timing
27. **CSID change terminates transaction:** Changing CSID with CSAAT=1 triggers implicit transaction termination with proper timing margins

7.6 Error Detection and Interrupts

28. **Error event conditions:** Six error types (CMDERR, OVERFLOW, UNDERFLOW, CMDINVAL, CSIDINVAL, ACCESSINVAL) detected and halt IP until cleared
29. **Interrupt aggregation:** Two interrupt classes (ERROR, SPI_EVENT) with secondary enable registers for fine-grain masking
30. **Error status registers:** ERROR_STATUS with write-one-to-clear semantics; errors must be acknowledged before processing continues
31. **Event interrupt triggers:** IDLE, READY, RXFULL, TXEMPTY, RXWM, TXWM events generate interrupts only on state entry transitions

- 32. **ACCESSINVAL cannot be disabled:** Invalid byte-enable patterns always trigger error as system integrity check
- 33. **Interrupt timing:** Status-type interrupts asserted when condition becomes true; cleared when software writes to INTR_STATE

7.7 Register Access and Memory Windows

- 34. **TXDATA memory window:** 32-bit word access with byte-enable support; byte/half-word writes consume full FIFO word with masking
- 35. **RXDATA memory window:** 32-bit word reads; partial words zero-padded based on ByteOrder parameter when segment ends
- 36. **Unaligned write handling:** Only byte, half-word, full-word writes supported; other patterns trigger ACCESSINVAL error
- 37. **Register update timing:** Configuration changes take effect at segment boundaries when CSB deasserted, not immediately
- 38. **STATUS register fields:** ACTIVE, READY, TXQD, RXQD, BYTE-ORDER, and FIFO flags reflect current transaction state

7.8 Control and Reset

- 39. **SPIEN control:** IP enable/disable via CONTROL.SPIEN; transactions only proceed when enabled
- 40. **SW_RST behavior:** Clears internal state including FIFOs, CDCs, FSM, and shift register; registers unchanged
- 41. **OUTPUT_EN control:** Enables/disables SCK, CSB, SD output buffers for bus sharing; modeled as enable flag
- 42. **Reset synchronization:** Software must confirm FIFOs empty before releasing from reset due to CDC draining behavior

7.9 Pass-through Mode

- 43. **Pass-through enable:** Multiplexes control between SPI_HOST FSM and external SPI_DEVICE via passthrough_i signal
- 44. **CSB[0] limitation:** Pass-through mode uses only CSB[0] when NumCS > 1; other chip selects unavailable
- 45. **Signal routing abstraction:** Pass-through modeled as control transfer, not detailed signal multiplexing

7.10 Excluded from TLM Model

- 46. **Physical layer characteristics:** Voltage levels, drive strength, impedance, capacitance not modeled
- 47. **Signal timing specifications:** Setup/hold times, propagation delays, rise/fall times excluded
- 48. **Pin multiplexing and integration:** System-level pin routing, base addresses, clock domains not modeled

49. **Electrical behavior:** Power consumption, signal integrity, EMI characteristics excluded
50. **RTL implementation details:** Internal pipeline stages, CDC synchronizer depth, exact FSM cycle counts abstracted

7.11 Summary

These assumptions establish a **transaction-level abstraction** suitable for **software-visible functional verification** of the SPI_HOST IP. The model focuses on:

- Command segmentation and execution as atomic operations
- FIFO management with depth, watermarks, and stall conditions
- Multi-device support via CSID and per-device configuration
- Error detection and interrupt generation for driver validation
- Register-level control and status visibility

The model **explicitly excludes**:

- Cycle-accurate timing and pin-level signaling
- Physical and electrical characteristics
- RTL implementation artifacts

This abstraction level enables efficient system-level simulation while maintaining full functional accuracy for software driver development and testing.

8. Conclusion

This high-level design specification provides a comprehensive foundation for implementing the SPI_HOST IP in SystemC TLM2.0. The document clearly delineates the modeling scope, configuration parameters, interface specifications, register behavior, and underlying assumptions necessary for transaction-level modeling.

Key aspects of the design include:

- **Segment-based command architecture** enabling flexible multi-phase SPI transactions
- **Configurable FIFO depths** with watermark-based flow control
- **Multiple chip select support** with per-device configuration
- **Comprehensive error detection** and interrupt management
- **Pass-through mode** for integration with SPI_DEVICE modules
- **Transaction-level abstraction** suitable for software driver verification

The specification maintains clear separation between software-visible functional behavior (to be modeled) and physical/electrical characteristics (excluded from modeling), ensuring an efficient yet functionally accurate SystemC TLM implementation.